3

XPath Basics

Contents

Background

Introduction

This lesson provides an overview of the features that make up the XML Path Language (XPath).

Lesson objectives

This lesson is designed to help you to:

- Define XPath
- Explain the basic XPath features used to access document content
- Use XPath Visualizer
- Utilize XPath Syntax and Expressions
- Use XPath Expression Builder to build XPath expressions

What is XPath?

Introduction

XML Path Language (XPath) is a common syntax and semantics for functionality that shared between XSL Transformations and XPointer. The primary purpose of XPath is to address parts of an XML document. In support of this primary purpose, it also provides basic facilities for manipulation of strings, numbers, and booleans. XPath uses a compact, non-XML syntax to facilitate use of XPath within URIs and XML attribute values. XPath operates on the abstract, logical structure of an XML document, rather than its surface syntax.

XPath expressions can also represent numbers, strings, or Booleans, so XSLT style sheets carry out simple arithmetic for numbering and cross-referencing figures, tables, and equations. String manipulation in XPath allows XSLT perform tasks like making the title of a chapter uppercase in a headline, but mixed case in a reference in the body text.

XPath Version

As of 2016 there are multiple versions of XPath. 3.0 being the latest version. Sterling IBM B2B Integrator supports XPath 1.0. This is still the most widely adopted of the versions.

The Structure of an XML Document

An XML document is a tree made up of nodes. Some nodes contain other nodes. One root node ultimately contains all other nodes. XPath is a language for picking nodes and sets of nodes out of this tree. There are seven kinds of nodes:

- The root node
- Element nodes
- Text nodes
- Attribute nodes
- Comment nodes
- Processing instruction nodes
- Namespace nodes

XPath Visualizer

Introduction

XPath Visualizer is a full blown visual XPath Interpreter for the evaluation of any XPath expression and visual presentation of the resulting nodeset or scalar value.

The XPath Visualizer's value as an XSLT and XPath learning and authoring tool results from its ability to present the results of any XPath expression in an immediate, appealing, and straightforward visualization. It allows the user to define their own dynamic xsl:variable and xsl:key elements and to use variable references and the key() functions in their XPath expressions.

A visual interpreter is good aid that can help you learn and test XPath queries and their results. It is not a necessary tool for the operation of Sterling B2B Integrator.

Note: Although this software is old as far as technology is considered it is one of the only XPath interpreters that only supports XPath version 1.0. Newer interpreters can also include newer versions of the XPath standard. This may or may not cause the results during testing to be different than the results in B2Bi.

Remember!	XPath Visualizer is a third-party software, and Sterling B2B Integrator Customer Support does not support it.

Exercise 3–1 Testing XPath Expressions using XPath Visualizer

Scenario

This exercise will have you test XPath expressions using XPath Visualizer.

Instructions

You can use the following steps to test XPath expressions in XPath Visualizer.

1. Go to **Data Files** folder in the desktop and open the **XPathVisualizer** folder.
2. Locate and click **xpathmain.html**.
3. When the page launches in IE click Enable Blocked Content at bottom of page to enable XPath Visualizer to run correctly.
4. Browse to **ClassLabfiles\Cookbook.xml** in the **Data Files** folder.
5. Click the **Select Node** to open the document.
6. Locate the **XPath Syntax and Expressions** section on the following page.
7. Type one or more XPath expressions from each section to learn about the different search capabilities.

Instructor Note: The XPath Visualizer is available in the Data Files. Ensure that you make this accessible to the participants.

XPath Syntax and Expressions

Introduction

XML Path, or XPath, is a language that is designed to operate on XML data. It is a powerful, easy language to learn. To explain XPath syntax and expressions, use the following XML document:

```
<?xml version="1.0" encoding="UTF-8"?>
<cookbook title="Easy Dinners My Kids Will Actually Eat">
     <notes>These are real recipes</notes>
    <recipe name="Shrimp Scampi">
        <ingredient>1 lb. peeled, deveined shrimp</ingredient>
        <ingredient>.5 cups melted butter</ingredient>
        <ingredient>.5 cups vegetable oil</ingredient>
        <ingredient>1 clove garlic, minced</ingredient>
        <ingredient>1 tblsp lemon juice</ingredient>
        <ingredient>dash cayenne pepper</ingredient>
        <notes info="cooking">Toss ingredients together and broil for 10
 minutes or until shrimp are cooked, stirring often.</notes>
        <sides>Rice or buttered noodles</sides>
    </recipe>
    <recipe name="Taco Pie">
        <ingredient>1 lb. Lean ground beef</ingredient>
        <ingredient>1 pkg. Corn bread or corn muffin mix</ingredient>
            <notes>Prepare as indicated on package</notes>
        <ingredient>1 pkt taco seasoning</ingredient>
<notes>Following instructions on seasoning pkt to prepare taco
             beef.</notes>
        <ingredient>2 cups shredded cheddar cheese</ingredient>
<notes info="cooking">Spread prepared beef in casserole dish. Blanket
with cheese. Cover with corn bread batter. Bake at 350 for 30 minutes,
or until cornbread is done.</notes>
        <sides>salad, fruit cocktail, etc.</sides>
</recipe>
    <ingredient>Sprig of parsley for garnish.</ingredient>
    <ingredient reason="just to make their eyes water">dash cayenne pepper</ingredient>
</cookbook>
```

(Continued on next page)

XPath Syntax and Expressions (Continued)

Locating Nodes

Example 1:

/cookbook/recipe/ingredient

This statement selects all "ingredient" elements within all "recipe" elements in the "cookbook" document. When a statement begins with a single "/", view the expression as searching an absolute path within the document.

Example 2:

//notes

This statement selects all "notes" elements in the document, regardless of their level. When an expression starts with "//", all nodes that are matching the criteria are selected throughout the document.

Using Wildcards

The following table lists few of the examples for using wildcards.

Example	Description
/cookbook/recipe/*	This statement selects all child elements of the "recipe" element within the "cookbook" document.
/cookbook/*/notes	This statement selects all "notes" elements that are grandchildren of the "cookbook" element. Hence, the first "notes" element would not be selected.
/*/*/notes	This statement selects all "notes" elements that have two ancestors.
//*	This statement selects all elements in the document, regardless of level.

(Continued on next page)

XPath Syntax and Expressions (Continued)

Selective Searching

Square brackets are used to specify an occurrence of a node within a document. Use parentheses when the argument is specified with the string characters instead of a number.

The following table lists few of the examples for selective searching.

Example	Description
/cookbook/recipe[2]	This statement selects the second "recipe" element within the "cookbook" element.
/cookbook/notes[last()]	This statement selects the last "notes" element within the "cookbook".
/cookbook/recipe[notes]	This statement selects all "recipe" elements in the cookbook that have a "notes" element within them.
/cookbook/recipe[ingredient='dash cayenne pepper']	This statement selects all "recipe" elements in the cookbook that include an "ingredient" element whose value is "dash cayenne pepper".
/cookbook/recipe[ingredient='dash cayenne pepper']/notes	This statement selects all "notes" elements of all "recipe" elements within the cookbook, that have an "ingredient" element whose value is "dash of cayenne pepper". Hence, the final "notes" node is not selected, as it is not part of a "recipe".

(Continued on next page)

XPath Syntax and Expressions (Continued)

Using the "And" Operator in Searches

The following table lists few examples for using the “And” operator in searches.

Example	Description
/cookbook/recipe/ingredient|/cookbook/recipe/notes	This statement selects all "ingredient" and all "notes" nodes of the "recipe" element within the cookbook.
//ingredient|//notes	This statement selects all "ingredient" and all "notes" elements in the cookbook.
//ingredient |//notes|//sides	This statement selects all "ingredient", all "notes", and all "sides" elements in the cookbook.
/cookbook/recipe/notes |//ingredient	This statement selects all "notes" elements within "recipe" nodes in the cookbook, as well as all "ingredient" elements in the entire document.

(Continued on next page)

XPath Syntax and Expressions (Continued)

Searching Attributes

You can search the attributes that are contained in a node by using the reserved character "@" as a prefix to the search argument. The following table lists few examples for searching attributes.

Example	Description
//@info	This statement selects all attributes named "info".
//recipe[@name]	This statement selects all "recipe" elements that have an attribute named "name".
//recipe[@*]	This statement selects all "recipe" elements that have an attribute of any name.
//recipe[@name='Taco Pie']	This statement selects all "recipe" elements that have an attribute named "name" that have a value of "Taco Pie".

Searching for Content

You can search a business process for a specific occurrence of a string within the data.

Example:

/cookbook/recipe [1]/ingredient [1]/text( )

This statement selects the string in the first ingredient in the first recipe. In this example, it returns the following string:

```
1 lb. peeled, deveined shrimp
```

(Continued on next page)

XPath Syntax and Expressions (Continued)

Numeric Expressions

XPath also support mathematical and numeric relational expressions, including equality and Boolean. Operators are as follows:

The following table lists the parameters that are shown in the previous example:

Operator	Description	Example
+	Addition	2+4
-	Subtraction	5-3
*	Multiplication	2*3
div	Division	12 div 6
mod	Modulus	7 mod 3 (remainder is the modulus)
=	Like or Equal	Cost = 10.00
!=	Not Equal	Cost != 50.00
<	Less Than	Cost < 50.00
<=	Less Than or Equal To	Cost <=49.00
>	Greater Than	Cost > 10.00
>=	Greater Than or Equal To	Cost >=11.00
or	Or	Cost >=11.00 or Cost <=49.00
and	And	Cost > 10.00 and Cost < 50.00
sum	Sums numeric values in elements	sum(Order/Cost)
count	Counts occurrences of specific nodes in data	count(Order/LineItem)

(Continued on next page)

XPath Syntax and Expressions (Continued)

Strings	Returns the value of an element and an object. It is helpful when the value you extract must be passed as a parameter to another service within a business process.
Substring	Substring allows you to locate and extract part of the content of an element without the entire element.
String-length	String-length returns an integer value that is number of bytes of data exist within an element.

Exercise 3–2 Viewing Results in XPath Visualizer

Scenario

You have received a purchase order from your customer in a standard XML format. Use XPath Visualizer to search, select, and perform simple manipulations with the data available in the purchase order.

Instructions

Follow the steps to use the XPath Visualizer.

1. Browse to **Lab Files** folder and select **Sample_Order.xml** using xPath visualizer.
2. Click **Process File** to open this xml document in xPath Visualizer.
3. Type each XPath expression to find the results:

- //Item/Price/text()
- string(//Item/Price)
- string(//Item[2]/Price)
- sum(//Line_Items/Item/Qty)
- count(//Line_Items/Item)
- (//Line_Items/Item[3]/Qty)*(//Line_Items/Item[3]/Price)
- //Item[@LI]
- /*/*/Item[@LI="2"]
- substring(/Purchase_Order/Ord_Num/text(),4,4)
- string-length(//Line_Items/Item[3]/Part_Num/text())
- /Purchase_Order/*/comment()[1]
- //Line_Items/*/*[local-name()='Price']/text()
- //Line_Items/*[local-name()=concat('ItemSplit_',//Line_Items/count/text())]/text()

Note: In the Sample_Order.xml file the Count value is declared as 2, so the second link 'http://www.supplier2.com' is selected. On changing the value of the 'Count' in the Sample_Order.xml file the corresponding link is selected. Changing the Count value can be automated by loop functionality of Repeat activity.

Instructor Note: Students to do this exercise, walk around and help where appropriate. Inform students that the local-name functionality returns the non-qualified data of the node specified. Also, inform that the comment functionality identifies the comments in the XML file. Make sure that all students complete this before continuing with next topic.

XPath Expression Builder

Overview

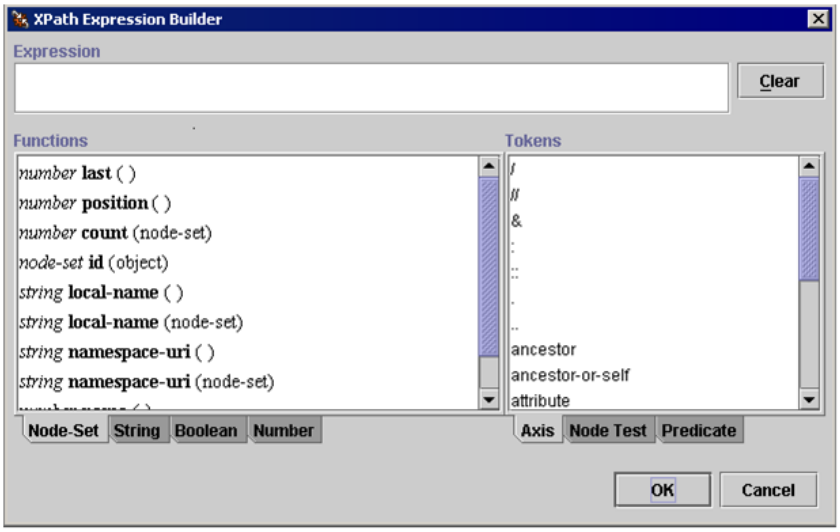
The XPath Expression Builder allows you to:

- Easily create complex rules within a business process; for example, a rule to locate a specific node or source content within an XML document.
- Create and define your own name and value for sending the name-value pairs through business process data.
- Describe the path for sending name-value pairs.

The XPath Expression Builder supports:

- Four expression types: Node-Set, String, Boolean, and Number
- Three token sets: Axis, Node Test, and Predicate

The following figure shows the XPath Expression Builder. The functions for the Node-Set expression type and the Axis token set are visible.



(Continued on next page)

XPath Expression Builder (Continued)

Expression Types

The following table describes the functions of the four expression types:

Expression Type	Function
Node-Set	■ Defines a set of sequential context-nodes that form a complete location path. ■ Specifies what information to process or return for the set of context-nodes.
String	■ Performs basic string operations, such as finding the length of a string, comparing a string, or changing letters from upper- to lower case. ■ Modifies the text content of XML elements or attributes. ■ Converts an argument of any type.
Boolean	Specifies a Boolean argument (using either the true or false state) that can include predicates and return the result.
Number	■ Specifies a numeric function for summing groups of numbers. ■ Finds the nearest integer to a number. ■ Performs basic arithmetic operations.

(Continued on next page)

XPath Expression Builder (Continued)

Token Sets

The following table describes the three token sets:

Expression Type	Function
Axis	Defines the direction of your search by using keywords, including: ■ Attributes ■ Child elements (default) ■ Descendants of the child elements ■ Parent elements ■ Ancestor elements
Node Test	Indicates the name of the node for which to search. Insert the name of the node test after the resolution operator (::) that follows the axis.
Predicate	Inserts a relational operator to test each node in the set of context-nodes of a location path. Predicates can employ Boolean operators.

(Continued on next page)

XPath Expression Builder (Continued)

Displaying XPath Expression Builder

After starting the GPM, you can open the XPath Expression Builder through the following features:

- Rule Manager – Assign rules and conditions.
- Service editor – Assign name-value pairs that represent source content and location in business process data.

Quiz

Questions

1. "/ProcessData/Order/OrderNumber" is an example of:
   a. An absolute path
   b. A relative path
2. In xPath, "/text()" and "string" perform the same function
   a. True
   b. False
3. Which of the following is an example of an indexed search using XPath?
   a. "/ProcessData/Order/item/text()"
   b. "/ProcessData/Order/item[3]/price"
   c. "/ProcessData/Order/*"
   d. "/ProcessData/Order/item"
4. Which reserved character is used to denote an attribute in XPath searches?
   a. *
   b. $
   c. @
   d. &

Lesson Review

Completed Objective

This lesson was designed to help you to:

- Define XPath
- Explain the basic XPath features used to access document content
- Use XPath Visualizer
- Utilize XPath Syntax and Expressions
- Use XPath Expression Builder to build XPath expressions

